Web Technologies Lab - 02

Aim:

This lab implements a client-side multi-step form validation system for an online job application portal using HTML, CSS, and JavaScript. Registration details are validated with regular expressions, and only on successful validation does the user proceed to the job application form.

Source Code:

File 1: register.html

```
<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <meta name="viewport" content="width=device-width, initial-scale=1.0">

 <title>Register</title>

 <style>

  :root {

   --bg: #f5efd1;

   --card: #ffffff;

   --ink: #1f2a37;

   --muted: #6b7280;

   --line: #d5dbe3;

   --accent: #3b5228;

   --accent-dark: #2f5885;

   --error: #c0392b;

   --ok: #2e7d5b;

  }

  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {

   font-family: "Segoe UI", system-ui, sans-serif;

   background: var(--bg);
```

```
  color: var(--ink);
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 1.5rem;
}
.card {
  background: var(--card);
  width: 100%;
  max-width: 440px;
  border: 1px solid var(--line);
  border-radius: 14px;
  padding: 2.25rem 2rem;
  box-shadow: 0 10px 30px rgba(31, 42, 55, 0.08);
}
h1 {
  font-size: 1.55rem;
  margin-bottom: 0.25rem;
}
.sub {
  color: var(--muted);
  font-size: 0.9rem;
  margin-bottom: 1.75rem;
}
.field { margin-bottom: 1.15rem; }
label {
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
```

```
  margin-bottom: 0.4rem;
}
input {
  width: 100%;
  padding: 0.65rem 0.75rem;
  font-size: 0.95rem;
  border: 1px solid var(--line);
  border-radius: 8px;
  outline: none;
  transition: border-color 0.15s ease, box-shadow 0.15s ease;
}
input:focus {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px rgba(59, 110, 165, 0.15);
}
input.invalid { border-color: var(--error); }
input.valid { border-color: var(--ok); }
.msg {
  font-size: 0.78rem;
  margin-top: 0.35rem;
  min-height: 1rem;
  color: var(--error);
}
/* password field with show/hide toggle */
.pw-wrap { position: relative; }
.pw-wrap input { padding-right: 3.2rem; }
.toggle {
  position: absolute;
  right: 0.6rem;
  top: 50%;
```

```
    transform: translateY(-50%);
    background: none;
    border: none;
    color: var(--accent);
    font-size: 0.78rem;
    font-weight: 600;
    cursor: pointer;
  }
  button.submit {
    width: 100%;
    margin-top: 0.5rem;
    padding: 0.75rem;
    background: var(--accent);
    color: #ecba98;
    border: none;
    border-radius: 8px;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.15s ease;
  }
  button.submit:hover { background: var(--accent-dark); }
  .hint {
    font-size: 0.72rem;
    color: var(--muted);
    margin-top: 0.3rem;
    line-height: 1.4;
  }
 </style>
</head>
```

```
<body>
 <div class="card">
  <h1>Welcome to JobFinder </h1>
  <p class="sub">Fill in the details below to proceed with your job application!</p>

  <div class="field">
   <label for="fullName">Full name</label>
   <input type="text" id="fullName" autocomplete="name">
   <div class="msg" id="fullNameMsg"></div>
  </div>

  <div class="field">
   <label for="email">Email</label>
   <input type="text" id="email" autocomplete="email">
   <div class="msg" id="emailMsg"></div>
  </div>

  <div class="field">
   <label for="phone">Phone number</label>
   <input type="text" id="phone" autocomplete="tel">
   <div class="msg" id="phoneMsg"></div>
  </div>

  <div class="field">
   <label for="password">Password</label>
   <div class="pw-wrap">
    <input type="password" id="password" autocomplete="new-password">
    <button type="button" class="toggle" id="togglePw">Show</button>
   </div>
   <div class="msg" id="passwordMsg"></div>
```

```
    <div class="hint">Min 8 chars, with an uppercase, a lowercase, a digit and a special character.</div>
  </div>

  <div class="field">
   <label for="confirm">Confirm password</label>
   <div class="pw-wrap">
    <input type="password" id="confirm" autocomplete="new-password">
    <button type="button" class="toggle" id="toggleConfirm">Show</button>
   </div>
   <div class="msg" id="confirmMsg"></div>
  </div>

  <button type="button" class="submit" id="registerBtn">Register</button>
 </div>

 <script>
  // next form's URL
  const NEXT_FORM_URL = "form2.html";

  // Regex validation rules
  const rules = {
   // letters and spaces only
   name:  /^[A-Za-z][A-Za-z\s]*$/,
   // standard email: something@something.domain
   email: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
   // exactly 10 digits
   phone: /^\d{10}$/,
   // at least one lowercase, one uppercase, one digit, one special, min 8
   password: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$/
```

```
};

// helper: set a field's error/success state and message
function setState(input, msgEl, ok, message) {
  input.classList.remove("valid", "invalid");
  input.classList.add(ok ? "valid" : "invalid");
  msgEl.textContent = ok ? "" : message;
  return ok;
}

//  individual validators
function validateName(id, msgId, label) {
  const input = document.getElementById(id);
  const msg = document.getElementById(msgId);
  const val = input.value.trim();
  if (val === "") return setState(input, msg, false, label + " can't be empty.");
  if (!rules.name.test(val)) return setState(input, msg, false, label + " may only contain letters and spaces.");
  return setState(input, msg, true);
}

function validateEmail() {
  const input = document.getElementById("email");
  const msg = document.getElementById("emailMsg");
  const val = input.value.trim();
  if (val === "") return setState(input, msg, false, "Email can't be empty.");
  if (!rules.email.test(val)) return setState(input, msg, false, "Enter a valid email address.");
  return setState(input, msg, true);
}
```

```
  function validatePhone() {
    const input = document.getElementById("phone");
    const msg = document.getElementById("phoneMsg");
    const val = input.value.trim();
    if (val === "") return setState(input, msg, false, "Phone number can't be empty.");
    if (!rules.phone.test(val)) return setState(input, msg, false, "Enter a 10-digit phone number.");
    return setState(input, msg, true);
  }

  function validatePassword() {
    const input = document.getElementById("password");
    const msg = document.getElementById("passwordMsg");
    const val = input.value;
    if (val === "") return setState(input, msg, false, "Password can't be empty.");
    if (!rules.password.test(val)) {
      return setState(input, msg, false, "Password needs 8+ chars with upper, lower, digit & special character.");
    }
    return setState(input, msg, true);
  }

  function validateConfirm() {
    const input = document.getElementById("confirm");
    const msg = document.getElementById("confirmMsg");
    const val = input.value;
    const pw = document.getElementById("password").value;
    if (val === "") return setState(input, msg, false, "Please confirm your password.");
    if (val !== pw) return setState(input, msg, false, "Passwords do not match.");
    return setState(input, msg, true);
  }
```

```
// hide/show password
function wireToggle(btnId, inputId) {
  const btn = document.getElementById(btnId);
  const input = document.getElementById(inputId);
  btn.addEventListener("click", function () {
    const showing = input.type === "text";
    input.type = showing ? "password" : "text";
    btn.textContent = showing ? "Show" : "Hide";
  });
}
wireToggle("togglePw", "password");
wireToggle("toggleConfirm", "confirm");

// live validation as the user types
document.getElementById("fullName").addEventListener("blur", () => validateName("fullName",
"fullNameMsg", "Full name"));
document.getElementById("email").addEventListener("blur", validateEmail);
document.getElementById("phone").addEventListener("blur", validatePhone);
document.getElementById("password").addEventListener("blur", validatePassword);
document.getElementById("confirm").addEventListener("blur", validateConfirm);

// register button: validate everything, then redirect
document.getElementById("registerBtn").addEventListener("click", function () {
  // run all checks (use & not && so every message shows at once)
  const results = [
    validateName("fullName", "fullNameMsg", "Full name"),
    validateEmail(),
    validatePhone(),
    validatePassword(),
```

```
    validateConfirm()
  ];

  const allValid = results.every(Boolean);

  if (allValid) {
    // everything passed — go to the next form
    window.location.href = NEXT_FORM_URL;
  }
 });
</script>
</body>
</html>
```

File 2: form2.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Job Application</title>
 <style>
  :root {
   --bg: #f5efd1;
   --card: #ffffff;
   --ink: #1f2a37;
   --muted: #6b7280;
   --line: #d5dbe3;
   --accent: #3b5228;
   --accent-dark: #2f5885;
   --error: #c0392b;
```

```
  --ok: #2e7d5b;
}
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: "Segoe UI", system-ui, sans-serif;
  background: var(--bg);
  color: var(--ink);
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 1.5rem;
}
.card {
  background: var(--card);
  width: 100%;
  max-width: 520px;
  border: 1px solid var(--line);
  border-radius: 14px;
  padding: 2.25rem 2rem;
  box-shadow: 0 10px 30px rgba(31, 42, 55, 0.08);
}
h1 { font-size: 1.55rem; margin-bottom: 0.25rem; }
.sub {
  color: var(--muted);
  font-size: 0.9rem;
  margin-bottom: 1.75rem;
}
.field { margin-bottom: 1.15rem; }
label {
```

```
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
  margin-bottom: 0.4rem;
}
input, select, textarea {
  width: 100%;
  padding: 0.65rem 0.75rem;
  font-size: 0.95rem;
  font-family: inherit;
  color: var(--ink);
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 8px;
  outline: none;
  transition: border-color 0.15s ease, box-shadow 0.15s ease;
}
textarea { resize: vertical; min-height: 110px; line-height: 1.5; }
input:focus, select:focus, textarea:focus {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px rgba(59, 110, 165, 0.15);
}
.invalid { border-color: var(--error); }
.valid { border-color: var(--ok); }
.msg {
  font-size: 0.78rem;
  margin-top: 0.35rem;
  min-height: 1rem;
  color: var(--error);
}
```

```
.hint {
  font-size: 0.72rem;
  color: var(--muted);
  margin-top: 0.3rem;
  line-height: 1.4;
}
.counter { float: right; font-weight: 400; color: var(--muted); }
button.submit {
  width: 100%;
  margin-top: 0.5rem;
  padding: 0.75rem;
  background: var(--accent);
  color: #ecba98;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.15s ease;
}
button.submit:hover { background: var(--accent-dark); }
.success {
  display: none;
  margin-top: 1.25rem;
  padding: 0.85rem 1rem;
  background: #e8f5ef;
  border: 1px solid var(--ok);
  border-radius: 8px;
  color: var(--ok);
  font-weight: 600;
```

```
    text-align: center;
   }
   .success.show { display: block; }
  </style>
</head>
<body>
  <div class="card">
   <h1>Job Application Form</h1>
   <p class="sub">Tell us about the role you want and what you bring to it.</p>

   <div class="field">
    <label for="position">Position</label>
    <select id="position">
     <option value="">-- Select a position --</option>
     <option value="frontend">Frontend Developer</option>
     <option value="backend">Backend Developer</option>
     <option value="fullstack">Full Stack Developer</option>
     <option value="data">Data Analyst</option>
     <option value="ml">Machine Learning Engineer</option>
     <option value="qa">QA Engineer</option>
    </select>
    <div class="msg" id="positionMsg"></div>
   </div>

   <div class="field">
    <label for="level">Job Level</label>
    <select id="level">
     <option value="">-- Select a level --</option>
     <option value="intern">Intern</option>
     <option value="entry">Entry Level</option>
```

```
      <option value="mid">Mid Level</option>
      <option value="senior">Senior</option>
      <option value="lead">Lead / Manager</option>
    </select>
    <div class="msg" id="levelMsg"></div>
  </div>

  <div class="field">
    <label for="experience">Experience (years)</label>
    <input type="text" id="experience" autocomplete="off">
    <div class="msg" id="experienceMsg"></div>
    <div class="hint">Enter your number of years of experience.</div>
  </div>

  <div class="field">
    <label for="skills">Skills</label>
    <input type="text" id="skills" autocomplete="off">
    <div class="msg" id="skillsMsg"></div>
    <div class="hint">Comma separated, e.g. HTML, CSS, JavaScript, Python.</div>
  </div>

  <div class="field">
    <label for="projects">Describe Your Projects <span class="counter" id="projCount">0 /
50</span></label>
    <textarea id="projects"></textarea>
    <div class="msg" id="projectsMsg"></div>
    <div class="hint">At least 50 characters.</div>
  </div>

  <div class="field">
```

```
  <label for="resume">Resume Upload</label>
  <input type="file" id="resume" accept=".pdf,.docx">
  <div class="msg" id="resumeMsg"></div>
  <div class="hint">PDF or DOCX only.</div>
 </div>

 <button type="button" class="submit" id="submitBtn">Submit Application</button>
 <div class="success" id="successBox">Application Submitted Successfully!</div>
</div>

<script>
 // validation rules
 const rules = {
  // whole number 0-30
  experience: /^\d{1,2}$/,
  // letters, digits, spaces, commas, +, #, ., -  (e.g. "C++, Node.js, HTML5")
  skills: /^[A-Za-z0-9+#.\s,-]+$/,
  // file must end in .pdf or .docx
  resume: /\.(pdf|docx)$/i
 };

 const MIN_PROJECT_CHARS = 50;

 // helper: set a field's error/success state and message
 function setState(el, msgEl, ok, message) {
  el.classList.remove("valid", "invalid");
  el.classList.add(ok ? "valid" : "invalid");
  msgEl.textContent = ok ? "" : message;
  return ok;
 }
```

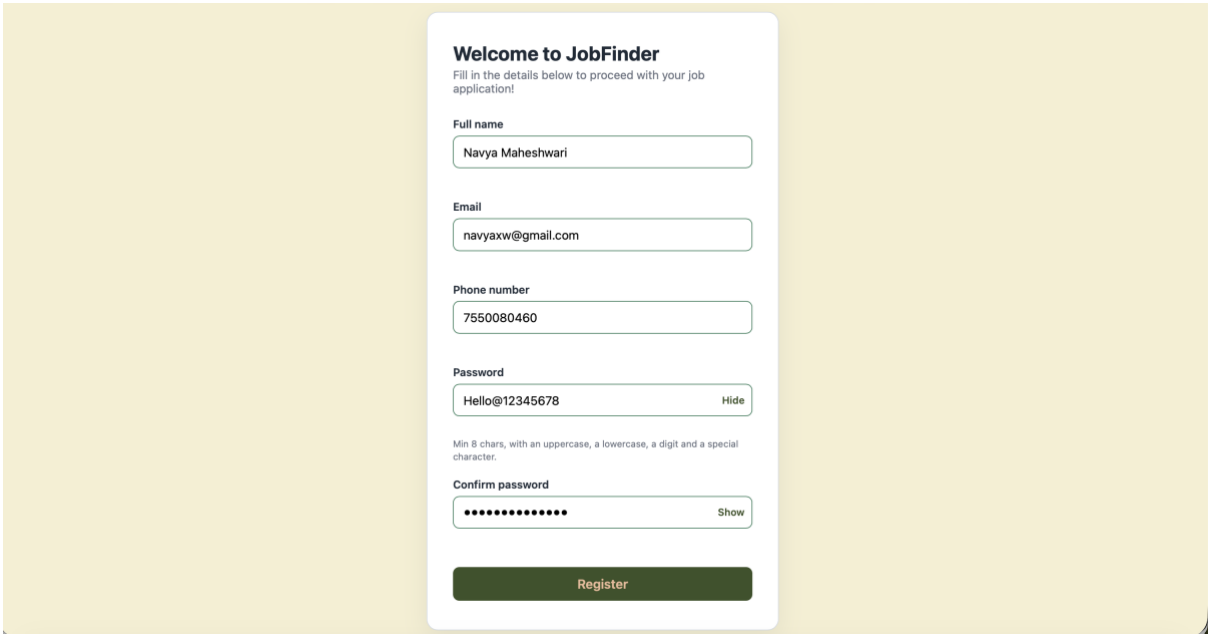
```
  // individual validators
  function validateSelect(id, msgId, label) {
    const el = document.getElementById(id);
    const msg = document.getElementById(msgId);
    if (el.value === "") return setState(el, msg, false, "Please select a " + label + ".");
    return setState(el, msg, true);
  }

  function validateExperience() {
    const el = document.getElementById("experience");
    const msg = document.getElementById("experienceMsg");
    const val = el.value.trim();
    if (val === "") return setState(el, msg, false, "Experience can't be empty.");
    if (!rules.experience.test(val)) return setState(el, msg, false, "Enter experience as a whole number.");
    const years = Number(val);
    if (years < 0 || years > 30) return setState(el, msg, false, "Experience must be between 0 and 30 years.");
    return setState(el, msg, true);
  }

  function validateSkills() {
    const el = document.getElementById("skills");
    const msg = document.getElementById("skillsMsg");
    const val = el.value.trim();
    if (val === "") return setState(el, msg, false, "Skills field can't be empty.");
    if (!rules.skills.test(val)) return setState(el, msg, false, "Skills may only contain letters, numbers and , + # . -");
    return setState(el, msg, true);
  }
```

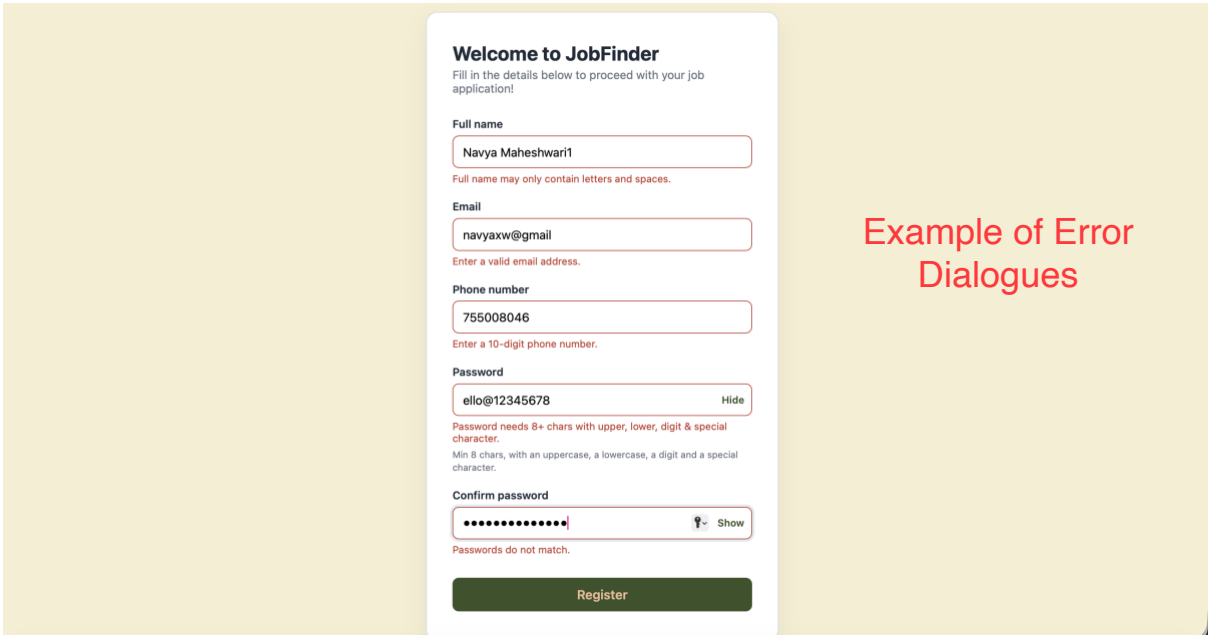
```
  function validateProjects() {
    const el = document.getElementById("projects");
    const msg = document.getElementById("projectsMsg");
    const val = el.value.trim();
    if (val === "") return setState(el, msg, false, "Project description can't be empty.");
    if (val.length < MIN_PROJECT_CHARS) {
      return setState(el, msg, false, "Please write at least " + MIN_PROJECT_CHARS + " characters (currently " + val.length + ").");
    }
    return setState(el, msg, true);
  }

  function validateResume() {
    const el = document.getElementById("resume");
    const msg = document.getElementById("resumeMsg");
    if (el.files.length === 0) return setState(el, msg, false, "Please upload your resume.");
    const fileName = el.files[0].name;
    if (!rules.resume.test(fileName)) return setState(el, msg, false, "Resume must be a PDF or DOCX file.");
    return setState(el, msg, true);
  }

  // ---------- Live character counter for the project box ----------
  const projInput = document.getElementById("projects");
  const projCount = document.getElementById("projCount");
  projInput.addEventListener("input", function () {
    projCount.textContent = projInput.value.trim().length + " / " + MIN_PROJECT_CHARS;
  });
```

```
  // ---------- Live validation ----------
  document.getElementById("position").addEventListener("change", () => validateSelect("position", "positionMsg", "position"));
  document.getElementById("level").addEventListener("change", () => validateSelect("level", "levelMsg", "job level"));
  document.getElementById("experience").addEventListener("blur", validateExperience);
  document.getElementById("skills").addEventListener("blur", validateSkills);
  document.getElementById("projects").addEventListener("blur", validateProjects);
  document.getElementById("resume").addEventListener("change", validateResume);

  // ---------- Submit: validate everything, then show success ----------
  document.getElementById("submitBtn").addEventListener("click", function () {
   // run all checks so every message shows at once
   const results = [
    validateSelect("position", "positionMsg", "position"),
    validateSelect("level", "levelMsg", "job level"),
    validateExperience(),
    validateSkills(),
    validateProjects(),
    validateResume()
   ];

   const allValid = results.every(Boolean);
   const box = document.getElementById("successBox");

   if (allValid) {
    box.classList.add("show");
    box.scrollIntoView({ behavior: "smooth", block: "nearest" });
   } else {
    box.classList.remove("show");
```

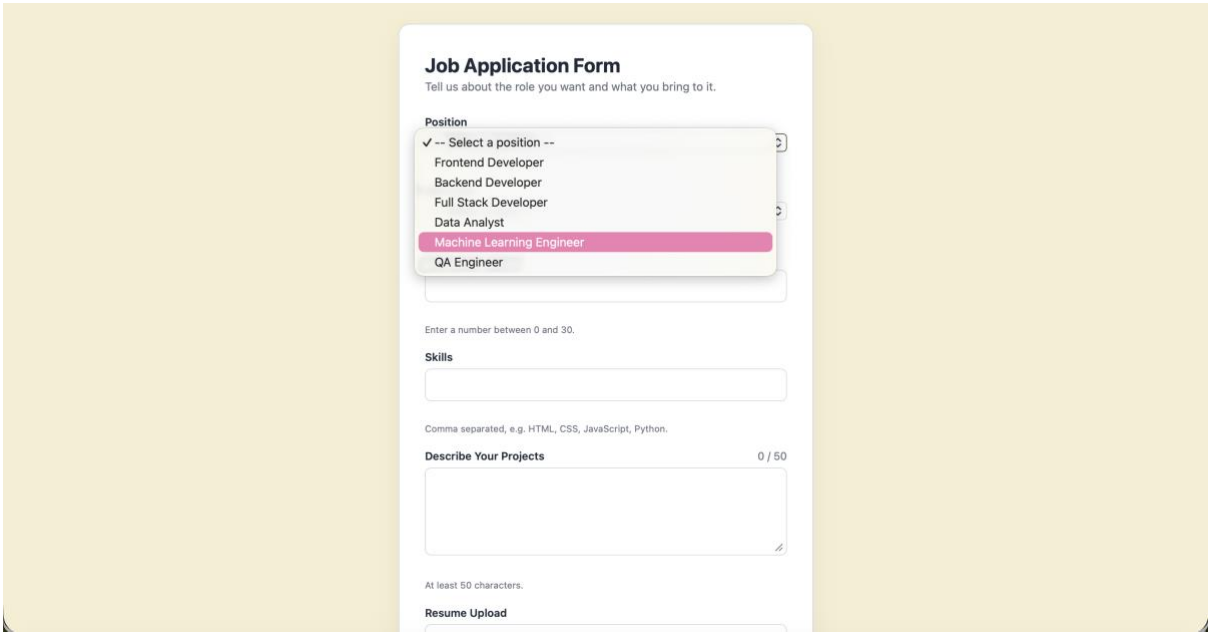
```
    }
  });
 </script>
</body>
</html>
```

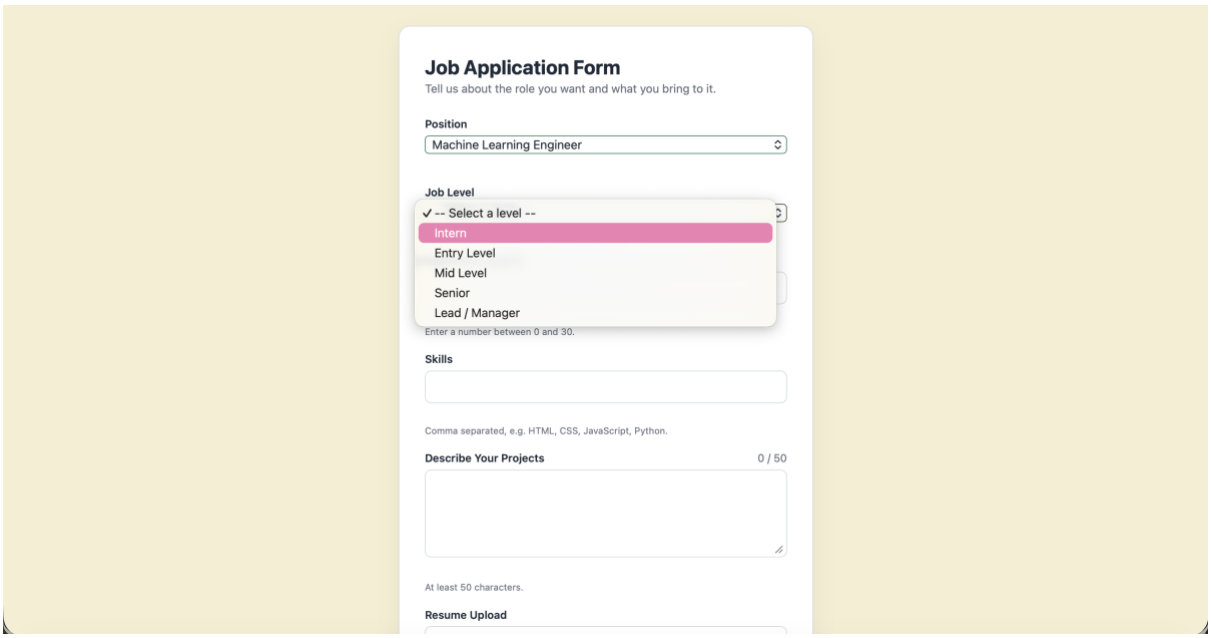
Output:

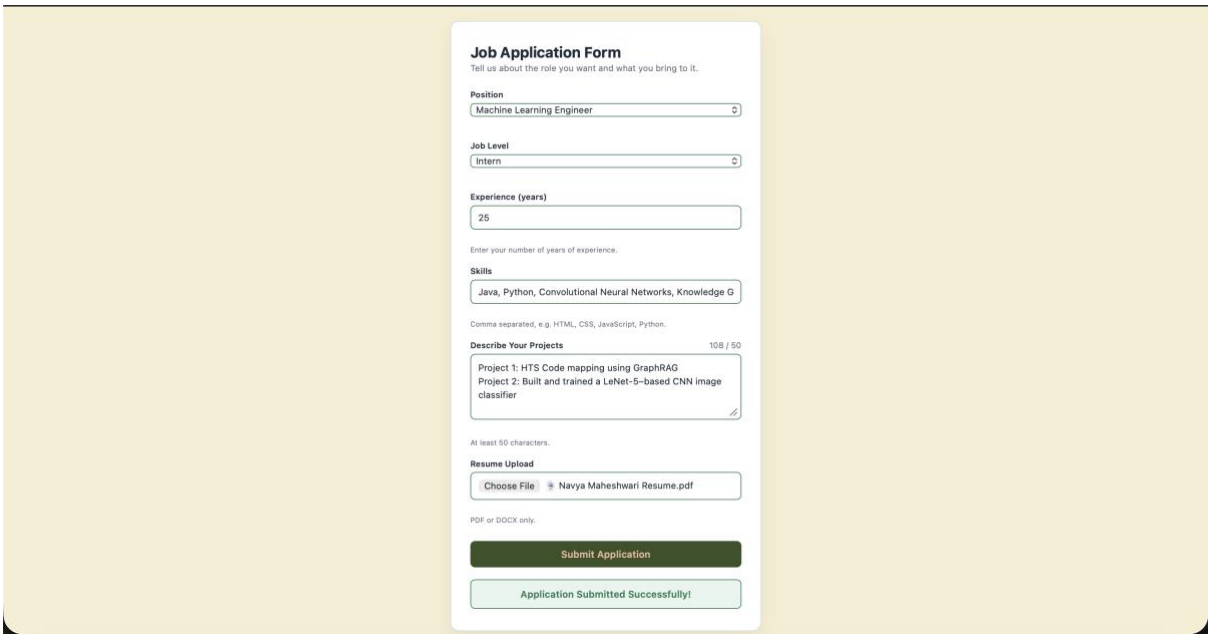




Example of Error Dialogues


Job Application Form
Tell us about the role you want and what you bring to it.
Position
-- Select a position --
Frontend Developer
Backend Developer
Full Stack Developer
Data Analyst
Machine Learning Engineer
QA Engineer
Enter a number between 0 and 30.
Skills
Comma separated, e.g. HTML, CSS, JavaScript, Python.
Describe Your Projects
0 / 50
At least 50 characters.
Resume Upload


Job Application Form
Tell us about the role you want and what you bring to it.
Position
Machine Learning Engineer
Job Level
-- Select a level --
Intern
Entry Level
Mid Level
Senior
Lead / Manager
Enter a number between 0 and 30.
Skills
Comma separated, e.g. HTML, CSS, JavaScript, Python.
Describe Your Projects
0 / 50
At least 50 characters.
Resume Upload



Conclusion:

All validation rules were implemented and tested successfully, with errors displayed inline and the application submitting only when every field passed. The exercise demonstrated how regular expressions and DOM event handling enforce data integrity before submission.